

76. Le monde glisse, le héros ne bouge pas

La position dans le monde vaut « défilement + 76 ».

1. Ce qu'il faut comprendre

Le lutin ne bouge **jamais** : il est à 76 pixels, image après image. Ce qui change, c'est `defiler()`.

C'est ce qui distingue un écran fixe d'un monde. La position du personnage **dans le monde** vaut `monde + 76`, et c'est ce nombre-là qu'on compare au décor — pas celui du lutin, qui est une constante.

La leçon « La caméra : c'est le monde qui bouge » part du même principe et le pousse jusqu'au bord du niveau.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1460 octets** de cartouche.

1. UN DESSIN — SOL

```
Tuile SOL = {  
    "#####", "-----", "-----", "-----",  
    "-----", "-----", "-----", "-----",  
};
```



à l'écran

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

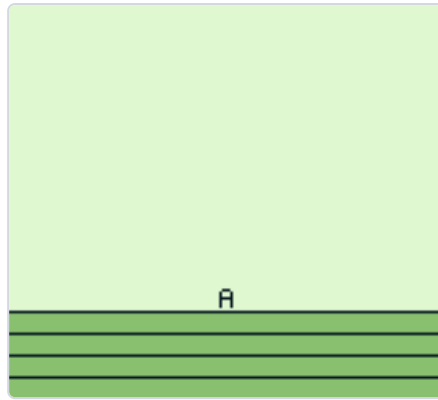
```
int main() {  
    for (uint8_t x = 0; x < 20; x++)  
        for (uint8_t y = 14; y < 18; y++)  
            poser(x, y, SOL);  
  
    uint8_t monde = 0;  
  
    while (true) {  
        if (bouton(DROITE)) monde++;  
        if (bouton(GAUCHE)) monde--;  
  
        defiler(monde, 0);  
        sprite(0, 76, 104, 1);  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



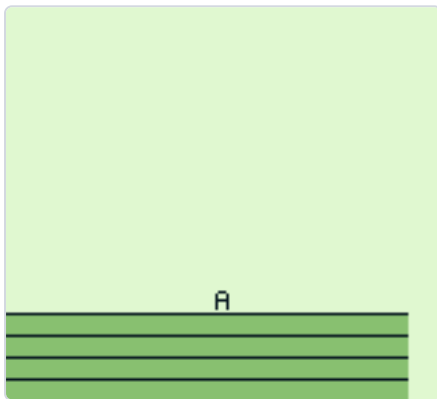
la toute première image — la console vient de démarrer — image 1



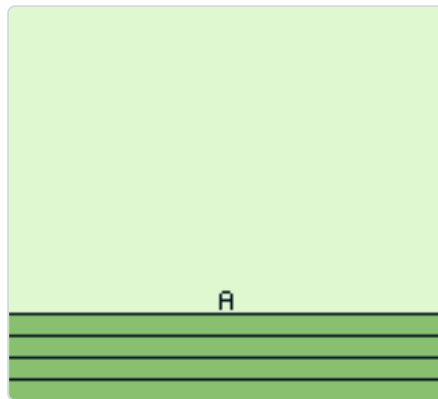
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé

5. Ce qu'on doit voir

Le décor glisse ; le personnage reste rigoureusement au milieu.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le monde a glissé ($0 \rightarrow 40$)
- ✓ le lutin n'a pas bougé d'un pixel ($x = 76$)
- ✓ il est resté au milieu
- ✓ le sol est posé

7. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

Game Boy en C++

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

SOL

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuances

Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

SOL

tuile n° 44 — poser(x, y, SO

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile

taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

SOL ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

SOL peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre... ▼